



Graph Neural Networks Powered by Encoder Embedding for Improved Node Clustering

Shiyu Chen, Youngser Park, Cencheng Shen, Carey E. Priebe

Johns Hopkins University, Department of Applied Mathematics and Statistics

University of Delaware, Department of Applied Economics and Statistics

schen355@jh.edu, youngser@jhu.edu, shenc@udel.edu, cep@jhu.edu



Abstract

We introduce a GEE-driven GNN (GG), which employs One-Hot Graph Encoder Embedding (GEE) to provide structured and expressive initialization for the GNN. We evaluate GG on node clustering tasks and simulations show that it converges faster and achieves superior results on certain graphs. Moreover, experiments on real-world datasets further demonstrate GG's strong performance, highlighting its potential as a powerful tool for diverse graph-related applications.

1. Method

1.1 GCN

We adopt the classic Graph Convolutional Network (GCN)[1] whose layer-wise propagation rule is defined as follows:

$$Z^{(l+1)} = GCN(Z^{(l)}, A) = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} Z^{(l)} W^{(l)} \right), \quad (1)$$

Here, $\tilde{A} = A + I_N$ is the adjacency matrix with added self-connections and $\tilde{D}$ of is the degree matrix based on $\tilde{A}$. $W^{(l)}$ represents the trainable weight matrix for the l -th layer, and $\sigma(\cdot)$ denotes the activation function. Finally, $Z^{(l)} \in R^{n \times d}$ is the activation matrix in the l -th layer, where $Z^{(0)}$ can either be randomized initial embeddings or specific node features. Here, d denotes the embedding dimension, which is set to match the number of clusters k .

To obtain the final representation $\hat{Z}$, we add the initial features to the outputs of each GCN layer[2], which can be formulated as $\hat{Z} = \sum_{l=0}^L Z^{(l)}$, where $L = 2$ denotes the number of layers.

Then, we use the loss of Deep Modularity Networks (DMoN)[3] to train our GNN, which is formulated as:

$$\mathcal{L}_{DMoN}(C; A) = -\frac{1}{2m} \text{Tr}(C^T B C) + \frac{\sqrt{k}}{n} \left\| \sum_i C_i^T \right\|_F - 1, \quad (2)$$

where $C = \text{softmax}(\hat{Z}) \in R^{n \times k}$ is the soft cluster assignment matrix, obtained by applying the softmax function to the GNN output $\hat{Z}$. The adjacency matrix of the graph is denoted as A , and $m = \frac{1}{2} \sum_{i,j} A_{ij}$ is the total number of edges. The modularity matrix B is defined as $B = A - \frac{v v^T}{2m}$, where v represents the degree vector of the graph.

Since the soft cluster assignment matrix C serves as a probability matrix, we assigned each node to the cluster with the highest probability for prediction:

$$\hat{Y} = \text{argmax}(C, \text{dim} = 1), \quad (3)$$

Finally, we can use a single function F_{GNN} to denote the entire computation of GNN with the output expressed as:

$$\hat{Y} = F_{GNN}(X_0, A) \quad (4)$$

1.2 GEE

GEE[4] is an efficient and flexible graph embedding method that can be applied with ground-truth labels, labels generated by other methods, partial labels, or without labels. The unsupervised version of GEE can be summarized as $\hat{Z}, \hat{Y} = F_{GEE}(E, k, M)$, where E denotes the edge list and M is the maximum number of iterations.

1.3 GEE-drivened GNN

We first generate a well-initialized node features $\hat{Z}_{GEE}$ from GEE, then utilize it as the input of the function F_{GNN} to obtain the predicted label vector $\hat{Y}_{GG}$:

$$\hat{Z}_{GEE}, \hat{Y}_{GEE} = F_{GEE}(E, k, M) \quad (5)$$

$$\hat{Y}_{GG} = F_{GNN}(\hat{Z}_{GEE}, A) \quad (6)$$

To compare the performance of GEE-drivened GNN, we generate the predicted label vector $\hat{Y}_{GNN}$ for the vanilla GNN, using a randomized initial embedding X_0 as the input of F_{GNN} :

$$\hat{Y}_{GNN} = F_{GNN}(X_0, A) \quad (7)$$

In our implementation, $X_0 \in R^{n \times k}$ is initialized using the Xavier uniform initialization, where each embedding element is sampled from $\mathcal{U} \left(-\sqrt{\frac{6}{k+n}}, \sqrt{\frac{6}{k+n}} \right)$.

2. Simulations

We evaluate our methods on Stochastic Block Model (SBM)[5] and Degree-Corrected Stochastic Block Model (DC-SBM)[6]. In both settings, we use $k = 4$ clusters with balanced cluster sizes, where each cluster contains n/k nodes.

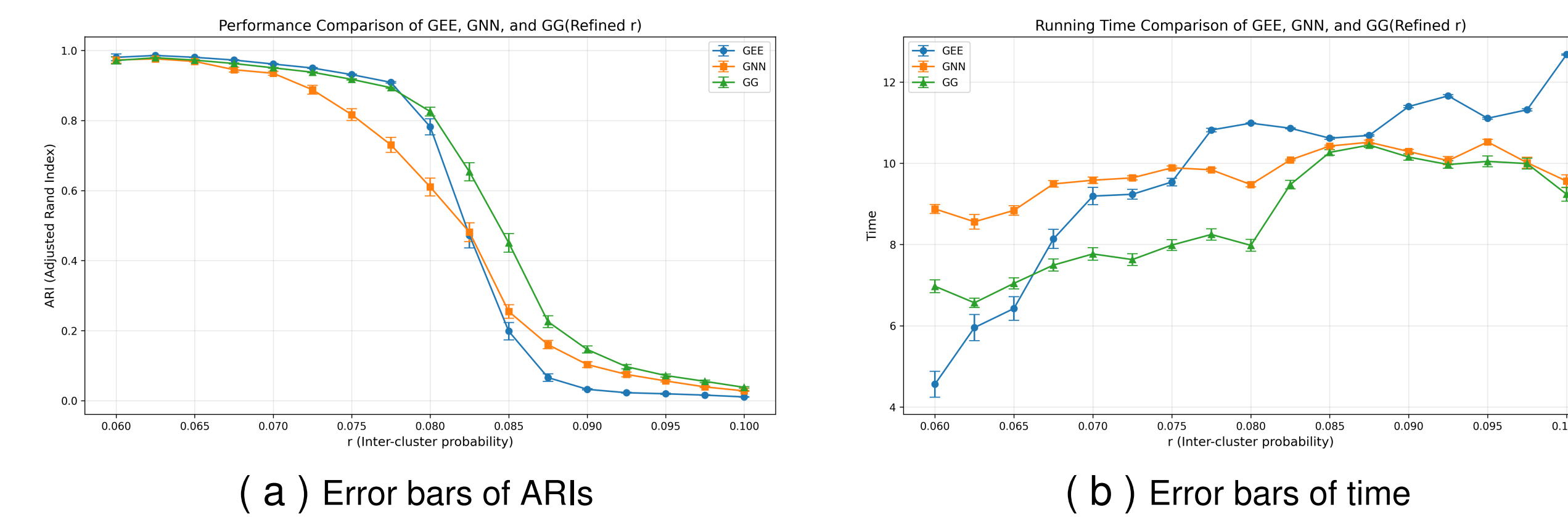


Figure 1: Error bars of ARIs and time for GEE, GNN, and GG based on 100 runs for each r ranging from 0.06 to 0.1 in steps of 0.0025.

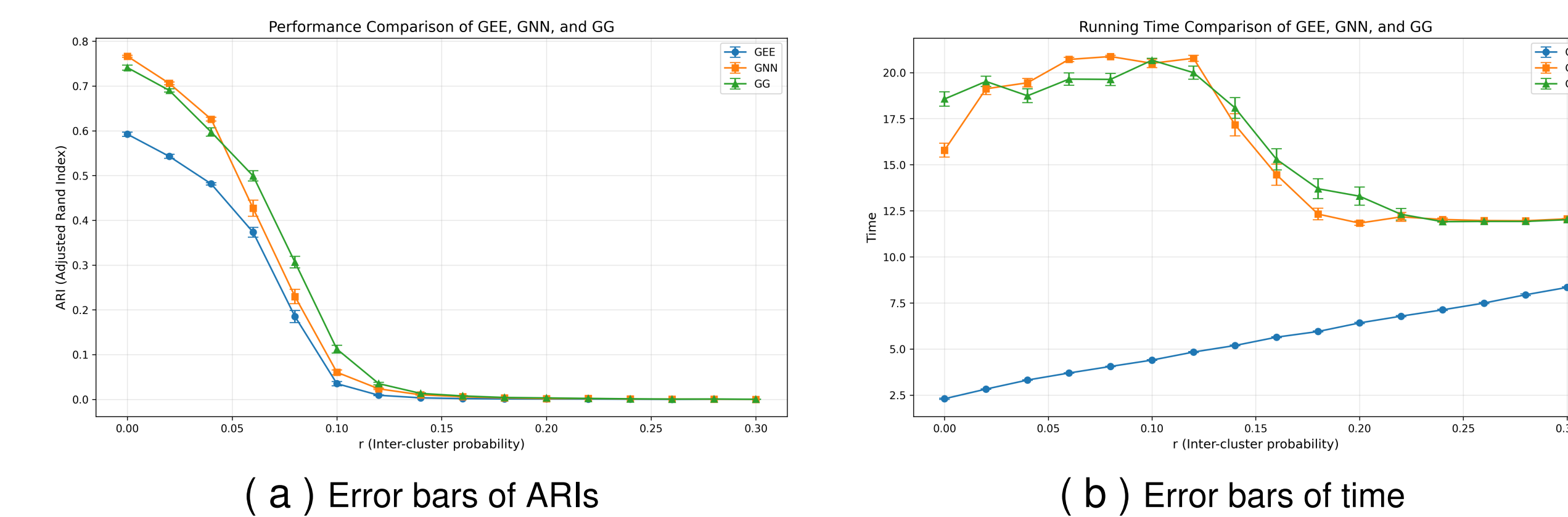


Figure 2: Error bars of ARIs and time for GEE, GNN, and GG based on 50 runs for each r ranging from 0 to 0.3 in steps of 0.02.

For SBM, we set $n = 800$ and within-cluster connection probability to 0.15. The between-cluster probabilities vary from 0.06 to 0.1 in steps of 0.0025. For DC-SBM, we set $n = 2000$

and within-cluster probability to 0.3. The between-cluster probabilities range from 0 to 0.3 in steps of 0.02. Then we depict the error bars of Adjusted Rand Index (ARI) and running time in Figure 1 and Figure 2, respectively. To better compare the convergence time of GNN with and without node features, we exclude the time of GEE when calculating the running time of GG, focusing solely on the convergence time of the GNN with GEE-generated node features.

3. Real Data Experiments

Finally, we conduct experiments on real graph data: Cora (2708 vertices, 5429 edges, 7 classes), Citeseer (3327 vertices, 4732 edges, 6 classes), Chameleon (2277 vertices, 18050 edges, 6 classes), ACM (3025 vertices, 13128 edges, 3 classes), EAT (399 vertices, 203 edges, 4 classes) and Industry (219 vertices, 630 edges, 3 classes). To mitigate the impact of randomness, we conduct 50 experiments on each dataset and calculate the mean values along with their standard errors. Given that the standard errors are relatively small, we use the mean values to plot the group bars, which are shown in Figure 3.

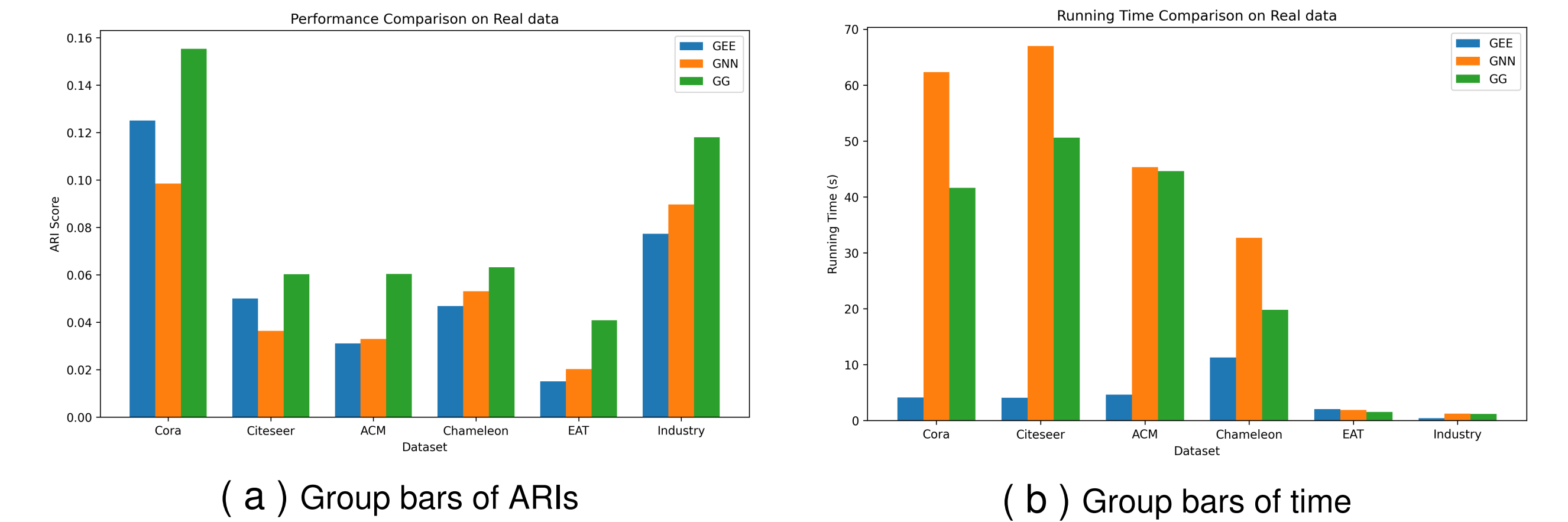


Figure 3: Group bars of ARIs and time on real data

4. Conclusions

We propose a simple, fast, and effective framework termed the GEE-driven Graph Neural Network (GG). Through extensive simulations and real-world data experiments, we demonstrate its superiority in generating more expressive representations and reducing convergence time. Despite the promising potential of GG, further exploration and theoretical analysis are required to identify the specific types of graphs where GG outperforms alternative approaches and to provide a deeper understanding of its underlying mechanisms.

References

- [1] Thomas N Kipf and Max Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [2] Guohao Li, Matthias Muller, Ali Thabet, and Bernard Ghanem, "Deepgcn: Can gcn go as deep as cnns?," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9267–9276.
- [3] Anton Tsitsulin, John Palowitch, Bryan Perozzi, and Emmanuel Müller, "Graph clustering with graph neural networks," *Journal of Machine Learning Research*, vol. 24, no. 127, pp. 1–21, 2023.
- [4] Cencheng Shen, Qizhe Wang, and Carey E Priebe, "One-hot graph encoder embedding," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 6, pp. 7933–7938, 2022.
- [5] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt, "Stochastic blockmodels: First steps," *Social networks*, vol. 5, no. 2, pp. 109–137, 1983.
- [6] Yunpeng Zhao, Elizaveta Levina, and Ji Zhu, "Consistency of community detection in networks under degree-corrected stochastic block models," 2012.